

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашнему заданию
«TODO менеджер задач на GO»

Выполнил:
Студент группы ИУ5-34Б
Яхутль М. А.
Преподаватель:
Нардид А. Н.

Москва 2025

Цель домашнего задания: изучение ранее неизвестного языка программирования.

Задание:

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Код main.go

```
package main

import (
    "bufio"    // для чтения ввода пользователя
    "encoding/json"
    "fmt"
    "os"
    "strconv"  // для конвертации строк в числа
    "strings"
    "time"
)

type Task struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Description string `json:"description"`
    Completed bool   `json:"completed"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

// динамический массив задач
type TaskList []Task

// файл для сохранения задач
const filename = "tasks.json"
```

```
func main() {  
    fmt.Println("Добро пожаловать в TODO менеджер!")  
    fmt.Println("=====  
  
    tasks := loadTasks()  
  
    for {  
        showMenu()  
  
        choice := getUserInput("Введите номер команды: ")  
        switch choice {  
        case "1":  
            showAllTasks(tasks)  
        case "2":  
            tasks = addTask(tasks)  
        case "3":  
            tasks = completeTask(tasks)  
        case "4":  
            tasks = deleteTask(tasks)  
        case "5":  
            saveTasks(tasks)  
            fmt.Println("Задачи сохранены!")  
        case "6":  
            saveTasks(tasks)  
            fmt.Println("До свидания!")
```

```

        return

    default:
        fmt.Println("Неверная команда, попробуйте снова")
    }

    fmt.Println() // пустая строка для читаемости
}

func showMenu() {
    fmt.Println("\n===== МЕНЮ =====")
    fmt.Println("1. Показать все задачи")
    fmt.Println("2. Добавить новую задачу")
    fmt.Println("3. Отметить задачу как выполненную")
    fmt.Println("4. Удалить задачу")
    fmt.Println("5. Сохранить задачи в файл")
    fmt.Println("6. Выход")
    fmt.Println("=====")
}

// читаем ввод пользователя с консоли
func getUserInput(prompt string) string {
    // prompt - текст подсказки, например "Введите название: "
    fmt.Print(prompt)

```

```
// создаем читатель для ввода с клавиатуры
reader := bufio.NewReader(os.Stdin)

// читаем строку
input, _ := reader.ReadString('\n')

// убираем лишние пробелы и символ новой строки
return strings.TrimSpace(input)
}

// загружаем задачи из JSON файла
func loadTasks() TaskList {
    file, err := os.Open(filename)
    if err != nil {
        //файл пустой
        fmt.Println("Файл задач не найден, начинаем с пустого списка")
        return TaskList{}
    }
    defer file.Close()

    // создаем декодер JSON
    decoder := json.NewDecoder(file)
    var tasks TaskList

    // декодируем JSON в нашу структуру
```

```
err = decoder.Decode(&tasks)
if err != nil {
    fmt.Println("Ошибка при чтении файла:", err)
    return TaskList{}
}

fmt.Printf("Загружено %d задач из файла\n", len(tasks))
return tasks
}
```

```
func saveTasks(tasks TaskList) {
    file, err := os.Create(filename)
    if err != nil {
        fmt.Println("Ошибка при создании файла:", err)
        return
    }
    defer file.Close()

    encoder := json.NewEncoder(file)
    encoder.SetIndent("", " ")

    // кодируем задачи в JSON и пишем в файл
    err = encoder.Encode(tasks)
    if err != nil {
        fmt.Println("Ошибка при сохранении задач:", err)
    }
}
```

```
}  
}
```

```
func showAllTasks(tasks TaskList) {
```

```
    if len(tasks) == 0 {
```

```
        fmt.Println("Список задач пуст")
```

```
        return
```

```
    }
```

```
    fmt.Printf("\nВсего задач: %d\n", len(tasks))
```

```
    fmt.Println("=====")
```

```
    for i, task := range tasks {
```

```
        // Определяем статус задачи
```

```
        status := "❌ "
```

```
        if task.Completed {
```

```
            status = "✅ "
```

```
        }
```

```
        // Форматируем дату для удобного отображения
```

```
        date := task.CreatedAt.Format("02.01.2006 15:04")
```

```
        // Выводим задачу
```

```
        fmt.Printf("%d. %s %s\n", i+1, status, task.Title)
```

```
        fmt.Printf("  %s\n", task.Description)
```



```
    fmt.Printf("    Создано: %s\n", date)
    fmt.Printf("    ID: %d\n", task.ID)
    fmt.Println("-----")
}
```

```
completed := countCompleted(tasks)
fmt.Printf("Статистика: %d выполнено, %d не выполнено\n",
    completed, len(tasks)-completed)
}
```

```
func countCompleted(tasks TaskList) int {
    count := 0
    for _, task := range tasks {
        if task.Completed {
            count++
        }
    }
    return count
}
```

```
func addTask(tasks TaskList) TaskList {
    fmt.Println("\nДобавление новой задачи")

    title := getUserInput("Введите название задачи: ")
    if title == "" {
```

```
    fmt.Println("Название не может быть пустым")
    return tasks
}
```

```
description := getUserInput("Введите описание (необязательно): ")
```

```
newTask := Task{
    ID:      generateID(tasks),
    Title:   title,
    Description: description,
    Completed: false,
    CreatedAt: time.Now(),
    UpdatedAt: time.Now(),
}
```

```
tasks = append(tasks, newTask)
```

```
fmt.Printf("Задача '%s' добавлена (ID: %d)\n", title, newTask.ID)
return tasks
}
```

```
func generateID(tasks TaskList) int {
    maxID := 0
    // Ищем максимальный ID среди существующих задач
    for _, task := range tasks {
```

```

        if task.ID > maxID {
            maxID = task.ID
        }
    }
    return maxID + 1
}

```

```

func completeTask(tasks TaskList) TaskList {
    if len(tasks) == 0 {
        fmt.Println("Нет задач для отметки")
        return tasks
    }

```

```

    fmt.Println("\nОтметка задачи как выполненной")
    showAllTasks(tasks)

```

```

    input := getUserInput("Введите номер задачи для отметки: ")
    index, err := strconv.Atoi(input) // Конвертируем строку в число
    if err != nil {
        fmt.Println("Введите корректный номер")
        return tasks
    }

```

```

    if index < 1 || index > len(tasks) {
        fmt.Printf("Номер должен быть от 1 до %d\n", len(tasks))
    }

```

```
    return tasks
}
```

```
tasks[index-1].Completed = true
tasks[index-1].UpdatedAt = time.Now()
```

```
    fmt.Printf("Задача '%s' отмечена как выполненная\n", tasks[index-1].Title)
    return tasks
}
```

```
func deleteTask(tasks TaskList) TaskList {
    if len(tasks) == 0 {
        fmt.Println("Нет задач для удаления")
        return tasks
    }
```

```
    fmt.Println("\nУдаление задачи")
    showAllTasks(tasks)
```

```
    input := getUserInput("Введите номер задачи для удаления: ")
    index, err := strconv.Atoi(input)
    if err != nil {
        fmt.Println("Введите корректный номер")
        return tasks
    }
```

```
}
```

```
if index < 1 || index > len(tasks) {  
    fmt.Printf("Номер должен быть от 1 до %d\n", len(tasks))  
    return tasks  
}
```

```
// Сохраняем название для сообщения  
title := tasks[index-1].Title
```

```
// Удаляем задачу из слайса  
// Создаем новый слайс без удаляемого элемента  
tasks = append(tasks[:index-1], tasks[index:]...)
```

```
fmt.Printf("Задача '%s' удалена\n", title)  
return tasks  
}
```

Работа кода:

```
PS C:\Users\Milen\OneDrive\Документы\forgo> go run main.go
Добро пожаловать в TODO менеджер!
=====
Загружено 0 задач из файла

===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 2

Добавление новой задачи
Введите название задачи: test-task #1
Введите описание (необязательно):
Задача 'test-task #1' добавлена (ID: 1)

===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 1

Всего задач: 1
=====
1. ✖ test-task #1

    Создано: 25.12.2025 12:52
    ID: 1
-----
Статистика: 0 выполнено, 1 не выполнено

===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 3

Отметка задачи как выполненной

Всего задач: 1
=====
1. ✖ test-task #1

    Создано: 25.12.2025 12:52
    ID: 1
-----
Статистика: 0 выполнено, 1 не выполнено
Введите номер задачи для отметки: 1
Задача 'test-task #1' отмечена как выполненная
```

```
===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 1

Всего задач: 1
=====
1. ☒ test-task #1

    Создано: 25.12.2025 12:52
    ID: 1
-----
Статистика: 1 выполнено, 0 не выполнено

===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 5
Задачи сохранены!
```

```
===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 4

Удаление задачи

Всего задач: 1
=====
1. ☒ test-task #1

    Создано: 25.12.2025 12:52
    ID: 1
-----
Статистика: 1 выполнено, 0 не выполнено
Введите номер задачи для удаления: 1
Задача 'test-task #1' удалена

===== МЕНЮ =====
1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход
=====
Введите номер команды: 1
Список задач пуст
```

===== МЕНЮ =====

1. Показать все задачи
2. Добавить новую задачу
3. Отметить задачу как выполненную
4. Удалить задачу
5. Сохранить задачи в файл
6. Выход

=====

Введите номер команды: 6

До свидания!